

Phase 5

Project Development Phase

Phase 5: Project Development Phase

The FitTrack AI project follows a well-structured backend architecture that separates business logic, routing, database operations, middleware, and AI services into individual modules. This modular design improves code readability, maintainability, and scalability while making the project easier to debug and extend.

The backend is developed using **Node.js** and **Express.js**, with **MongoDB** as the primary database and **Google Gemini AI** for intelligent fitness recommendations.

Creating Project Folder

Step 1: Create the Project Folder

- Create a new folder with the project name: AI-FitTrack-API

Step 2: Open the Project

- Open the project folder in **Visual Studio Code**.

Step 3: Create the Server Folder

- Inside the project directory, create a folder named: **Server**

Step 4: Initialize Node Project

- Open the terminal inside the **server** folder and execute: **npm init -y**

This command generates the **package.json** file containing project metadata and dependency information.

Step 5: Install Required Packages

- Install the required dependencies using:
 - **npm install express mongoose dotenv bcryptjs jsonwebtoken cors**
- Install Nodemon as a development dependency using:
 - **npm install --save-dev nodemon**

Environment Variables

Sensitive information is stored in the **.env** file.

PORT=5000

MONGO_URI=mongodb://localhost:27017/aifittrack

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_google_gemini_api_key

Using environment variables improves security by preventing sensitive information from being exposed in source code.

Backend Development

Backend development focuses on implementing REST APIs that allow authenticated users to manage workouts and interact with Google Gemini AI.

The project is divided into multiple modules that collectively perform authentication, workout management, AI integration, and database operations.

Major modules include:

- Server Configuration
- Database Configuration
- Authentication
- Workout Management
- AI Recommendation
- Middleware
- Error Handling

Database Configuration

The FitTrack AI backend uses MongoDB as its primary NoSQL database to securely store user credentials and workout data, paired with Mongoose as the Object Data Modeling (ODM) library for seamless application-to-database interaction. Together, this database layer maintains data integrity and validates incoming requests while providing schema validation, middleware support, data relationships, and simplified CRUD operations.

MongoDB Configuration

MongoDB stores application data in collections rather than traditional relational database tables. The FitTrack AI maintains separate collections for users and workouts.

Database Collections

- Users Collection
- Workouts Collection

Each collection stores JSON-like documents (BSON format), allowing flexible schema design while maintaining efficient data retrieval.

Installing Mongoose

- Install Mongoose using: `npm install mongoose`

Mongoose acts as the bridge between the Node.js application and MongoDB by providing schema definitions, validation, and database methods.

MongoDB Connection

The database connection is configured inside the **config/db.js** file. This module establishes a connection to the MongoDB server when the application starts.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

- **Responsibilities:** Connects to MongoDB, handles connection errors, displays connection status, and exports the database connection.
- **Advantages:** Provides centralized database configuration, a reusable connection module, automatic connection management, and simplified error handling.

Mongoose Schemas

The AI FitTrack backend contains two primary Mongoose schemas:

1. User Schema
2. Workout Schema

Schemas define the structure, validation rules, and relationships for MongoDB documents.

1. User Schema (models/User.js)

The User schema stores authentication information for registered users.

```
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5    {
6      name: {
7        type: String,
8        required: [true, 'Please add a name'],
9        trim: true,
10     },
11     email: {
12       type: String,
13       required: [true, 'Please add an email'],
14       unique: true,
15       trim: true,
16       lowercase: true,
17       match: [
18         /^([a-zA-Z0-9_\.\-])+\@([a-zA-Z0-9_\.\-])+\.(a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z){2,3}$/,
19         'Please add a valid email address',
20       ],
21     },
22     password: {
23       type: String,
24       required: [true, 'Please add a password'],
25       minlength: [6, 'password must be at least 6 characters'],
26     },
27   },
28   {
29     timestamps: true,
30   }
31 );
32
33 // Pre-save password hashing hook
34 userSchema.pre('save', async function (next) {
35   if (!this.isModified('password')) {
36     return next();
37   }
38   const salt = await bcrypt.genSalt(10);
39   this.password = await bcrypt.hash(this.password, salt);
40   next();
41 });
42
43 // Compare password method
44 userSchema.methods.matchPassword = async function (enteredPassword) {
45   return await bcrypt.compare(enteredPassword, this.password);
46 };
47
48 module.exports = mongoose.model('User', userSchema);
```

Validation Rules

- Name cannot be empty.
- Email must be unique.
- Password is mandatory.
- Password is encrypted using bcrypt.js before saving.
- Automatic timestamps are enabled.

User Schema Features

- Email uniqueness validation
- Password hashing
- Automatic timestamps
- Secure authentication support
- JWT integration

2. Workout Schema (models/Workout.js)

The Workout schema stores workout information created by authenticated users.

```
src > models > JS Workout.js > workoutSchema > category > enum > values
1  const mongoose = require('mongoose');
2
3  const workoutSchema = new mongoose.Schema(
4    {
5      workoutName: {
6        type: String,
7        required: [true, 'Please add a workout name'],
8        trim: true,
9      },
10     category: {
11       type: String,
12       required: [true, 'Please select a workout category'],
13       enum: {
14         values: [
15           'Cardio',
16           'Strength Training',
17           'Yoga',
18           'Running',
19           'Cycling',
20           'Walking',
21         ],
22         message: '{VALUE} is not a supported workout category',
23       },
24     },
25     duration: {
26       type: Number,
27       required: [true, 'Please add the workout duration in minutes'],
28       min: [1, 'Duration must be at least 1 minute'],
29     },
30     caloriesBurned: {
31       type: Number,
32       required: [true, 'Please specify calories burned'],
33       min: [0, 'Calories burned cannot be negative'],
34     },
35     workoutDate: {
36       type: Date,
37       default: Date.now,
38     },
39     user: {
40       type: mongoose.Schema.Types.ObjectId,
41       ref: 'User',
42       required: true,
43     },
44   },
45   {
46     timestamps: true,
47   }
48 );
49
50 module.exports = mongoose.model('Workout', workoutSchema);
```

Validation Rules

- Workout Name is mandatory.
- Category is mandatory.
- Duration must be greater than zero.
- Calories Burned must be greater than or equal to zero.
- Workout Date is required.
- Every workout belongs to one authenticated user.

Workout Schema Features

- Schema validation
- Automatic timestamps
- User relationship
- Easy search support
- Efficient CRUD operation

Project Execution

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server: **npm start**

Code: [GitHub Repo Link](#)